LISTA DE EXERCÍCIOS

Fixação, prática, preparação para prova e desafios

Unidade 3 - Modelo de objetos, classes e instâncias

PHP 8.4

Conteúdo trabalhado

Abstração | Encapsulamento | public e private | Property Promotion | Propriedades | Métodos

Instruções de realização e entrega

A lista foi organizada de forma progressiva. Os primeiros exercícios são guiados e focados na mecânica de classes e objetos; gradualmente, as regras exigem mais lógica, proteção de estado e interpretação. Os exercícios finais simulam o estilo de questão em que o problema precisa ser modelado antes da implementação.

- A atividade é individual.
- O aluno pode criar um projeto PHP do zero ou utilizar o projeto base fornecido em sala.
- Cada classe deve ficar em seu próprio arquivo .php, com nome compatível com a classe.
- Cada exercício deve possuir seu próprio arquivo de teste: ex1.php, ex2.php, ex3.php, ... ex20.php. Não concentre os testes em um único index.php.
- No início de cada arquivo de teste, inclua um comentário com: nome completo, RA, turma e disciplina.
- Use tipos nas propriedades, parâmetros e retornos sempre que forem adequados ao problema.
- Utilize public somente quando a propriedade realmente puder ser acessada diretamente. Estados que dependem de regras devem ser private e alterados por métodos públicos.
- Use constructor property promotion quando o construtor apenas receber e armazenar uma propriedade. Não é obrigatório promover todas as propriedades.
- Encapsulamento não significa criar getters e setters para tudo. Prefira métodos que expressem comportamentos e preservem as regras do objeto.
- Não é necessário utilizar herança, interfaces, traits, protected, enums ou outros recursos ainda não estudados.
- Ao finalizar, compacte o projeto em um arquivo .zip e envie a entrega na atividade correspondente do Moodle.

Organização da lista

Exercícios 1 a 5	Fixação - classes pequenas, cálculos, estado e primeiros invariantes.
Exercícios 6 a 10	Prática - situações reais com alterações de estado e regras de negócio.
Exercícios 11 a 15	Preparação para prova - múltiplas condições e maior atenção ao encapsulamento.
Exercício 16	Transição - estrutura indicada, mas regras precisam ser interpretadas.
Exercícios 17 a 20	Desafios - cenário curto e modelagem pelo aluno, sem classe pronta no enunciado.

Exercício 1 - Retângulo [Fixação]

Classe: `Retangulo`

Propriedades:

- private float $largura
- private float $altura

Métodos e retornos:

- area(): float - retorna largura * altura.
- perimetro(): float - retorna 2 * (largura + altura).
- ehQuadrado(): bool - retorna true quando largura e altura forem iguais.

Regras:

- Receba largura e altura no construtor, preferencialmente com property promotion.
- As dimensões devem ser maiores que zero. Valores inválidos não devem deixar o objeto em um estado inconsistente.

Teste no ex1.php: crie pelo menos dois retângulos, sendo um deles um quadrado, e mostre área, perímetro e o resultado de ehQuadrado().

Exercício 2 - Temperatura controlada [Fixação]

Classe: `Temperatura`

Propriedades:

- private float $celsius

Métodos e retornos:

- alterar(float $novoValor): bool - altera a temperatura se o valor for válido.
- emFahrenheit(): float - retorna (celsius * 9 / 5) + 32.
- emKelvin(): float - retorna celsius + 273.15.
- descricao(): string - retorna uma descrição com as três escalas.

Regras:

- Não aceite valores inferiores a -273.15 °C.
- A propriedade celsius não pode ser modificada diretamente fora da classe.

Teste no ex2.php: teste uma temperatura comum, uma negativa e uma tentativa de alteração abaixo do zero absoluto.

Exercício 3 - Ingresso de cinema [Fixação]

Classe: `IngressoCinema`

Propriedades:

- public string $filme
- private float $precoBase
- private bool $meiaEntrada

Métodos e retornos:

- calcularValorFinal(): float - retorna o preço integral ou 50% do preço base.
- definirMeiaEntrada(bool $possuiDireito): void - altera a condição de meia-entrada.
- resumo(): string - retorna filme, tipo do ingresso e valor final.

Regras:

- Utilize property promotion no construtor para as propriedades que fizer sentido.
- precoBase deve ser maior que zero.
- O preço final deve ser calculado pelo método e não armazenado manualmente.

Teste no ex3.php: crie dois ingressos para o mesmo filme, um inteiro e um meia-entrada, e compare os valores.

Exercício 4 - Cronômetro de treino [Fixação]

Classe: `CronometroTreino`

Propriedades:

- public string $atividade
- private int $segundosAcumulados

Métodos e retornos:

- adicionarTempo(int $segundos): bool - soma apenas valores positivos.
- zerar(): void - redefine o tempo acumulado para zero.
- totalMinutos(): float - retorna o tempo em minutos.
- formatarTempo(): string - retorna o total no formato HH:MM:SS.

Regras:

- O cronômetro deve iniciar em zero.
- O tempo acumulado não pode ser alterado diretamente fora da classe.

Teste no ex4.php: adicione tempos em três etapas, exiba o formato final, zere o cronômetro e confirme o novo estado.

Exercício 5 - Triângulo e suas regras [Fixação]

Classe: `Triangulo`

Propriedades:

- private float $ladoA
- private float $ladoB
- private float $ladoC

Métodos e retornos:

- ehValido(): bool - verifica a desigualdade triangular.
- classificar(): string - retorna Equilátero, Isósceles, Escaleno ou Inválido.
- perimetro(): float - retorna a soma dos lados quando o triângulo for válido.

Regras:

- Todos os lados devem ser maiores que zero.
- Um lado deve ser menor que a soma dos outros dois para existir um triângulo.
- Evite permitir alteração direta dos lados após a criação do objeto.

Teste no ex5.php: teste ao menos um triângulo equilátero, um escaleno e um conjunto de lados que não forme triângulo.

Exercício 6 - Ticket de estacionamento [Prática]

Classe: `TicketEstacionamento`

Propriedades:

- public string $placa
- private int $entradaMin
- private ?int $saidaMin
- private float $tarifaHora

Métodos e retornos:

- registrarSaida(int $minuto): bool - registra a saída somente se for posterior à entrada.
- duracaoMin(): int - retorna a duração; antes da saída, pode retornar 0.
- valorAPagar(): float - cobra a tarifa por hora, arredondando qualquer fração para a próxima hora completa.
- resumo(): string - retorna placa, duração e valor.

Regras:

- Considere entrada e saída como minutos decorridos no mesmo dia.
- A saída deve ser registrada apenas uma vez.
- tarifaHora deve ser positiva.

Teste no ex6.php: simule permanências de 40, 60 e 125 minutos e verifique a cobrança.

Exercício 7 - Semáforo inteligente [Prática]

Classe: `Semaforo`

Propriedades:

- public string $local
- private string $cor
- private int $ciclosCompletos

Métodos e retornos:

- avancar(): void - muda a cor na sequência vermelho -> verde -> amarelo -> vermelho.
- podePassar(): bool - retorna true somente quando a cor for verde.
- estado(): string - retorna local, cor atual e quantidade de ciclos completos.

Regras:

- O semáforo deve iniciar em vermelho.
- A cor não pode ser alterada diretamente pelo código externo.
- Considere um ciclo completo quando o semáforo retornar de amarelo para vermelho.

Teste no ex7.php: execute pelo menos sete mudanças de cor e mostre o estado após cada avanço.

Exercício 8 - Lâmpada inteligente [Prática]

Classe: `LampadaInteligente`

Propriedades:

- public string $comodo
- private bool $ligada
- private int $intensidade

Métodos e retornos:

- ligar(): void - liga a lâmpada.
- desligar(): void - desliga a lâmpada.
- ajustarIntensidade(int $valor): bool - aceita valores de 0 a 100.
- status(): string - retorna cômodo, estado e intensidade.

Regras:

- A intensidade deve permanecer entre 0 e 100.
- A lâmpada inicia desligada e com intensidade 50.
- Uma tentativa de intensidade inválida não deve alterar o valor anterior.

Teste no ex8.php: crie duas lâmpadas, altere intensidade, ligue/desligue e tente informar valores inválidos.

Exercício 9 - Personagem de RPG [Prática]

Classe: `PersonagemRPG`

Propriedades:

- public string $nome
- public string $classe
- private int $vida
- private int $energia

Métodos e retornos:

- receberDano(int $pontos): bool - reduz a vida quando pontos > 0.
- curar(int $pontos): bool - aumenta a vida sem ultrapassar 100.
- usarHabilidade(int $custoEnergia): bool - consome energia apenas se houver energia suficiente e o personagem estiver vivo.
- descansar(int $pontos): bool - recupera energia sem ultrapassar 100.
- status(): string - retorna os valores atuais.

Regras:

- vida e energia variam de 0 a 100.
- Um personagem com vida 0 não pode usar habilidade.
- Nenhum método pode produzir valores negativos ou maiores que 100.

Teste no ex9.php: simule dano, cura, uso de habilidades, falta de energia e recuperação.

Exercício 10 - Cofrinho digital com meta [Prática]

Classe: `CofrinhoMeta`

Propriedades:

- public string $objetivo
- private float $saldo
- private float $meta

Métodos e retornos:

- depositar(float $valor): bool - adiciona valores positivos.
- retirar(float $valor): bool - retira somente quando houver saldo suficiente.
- percentualDaMeta(): float - retorna o percentual já alcançado.
- metaAtingida(): bool - retorna true quando saldo >= meta.
- resumo(): string - retorna objetivo, saldo, meta e progresso.

Regras:

- A meta deve ser maior que zero.
- saldo nunca pode ficar negativo.
- O percentual pode ultrapassar 100% caso o saldo supere a meta.

Teste no ex10.php: faça depósitos e retiradas válidas e inválidas até atingir a meta.

Exercício 11 - Cartão de transporte urbano [Preparação para prova]

Classe: `CartaoTransporte`

Propriedades:

- public string $titular
- private float $saldo
- private float $tarifa
- private int $viagensRealizadas

Métodos e retornos:

- recarregar(float $valor): bool - adiciona crédito positivo.
- embarcar(): bool - desconta uma tarifa se houver saldo suficiente.
- saldoAtual(): float - retorna o saldo.
- viagensRealizadas(): int - retorna a quantidade de embarques confirmados.
- resumo(): string - retorna os dados relevantes.

Regras:

- O cartão inicia com zero viagens.
- A tarifa deve ser positiva.
- Embarques sem saldo suficiente devem falhar sem alterar o estado.

Teste no ex11.php: recarregue o cartão, faça vários embarques até o saldo se tornar insuficiente e confirme o total de viagens.

Exercício 12 - Hidrômetro residencial [Preparação para prova]

Classe: `Hidrometro`

Propriedades:

- public string $identificacao
- private float $leituraAnterior
- private float $leituraAtual

Métodos e retornos:

- registrarLeitura(float $novaLeitura): bool - aceita apenas leituras maiores ou iguais à atual.
- consumoUltimoPeriodo(): float - retorna leituraAtual - leituraAnterior.
- estimarConta(float $precoPorM3): float - retorna consumo * preço por m³.
- resumo(): string - retorna leituras e consumo.

Regras:

- Ao registrar nova leitura, a leitura atual passa a ser a anterior e a nova leitura passa a ser a atual.
- Leituras e preço por m³ não podem ser negativos.
- Uma leitura menor que a atual deve ser recusada sem alterar o histórico.

Teste no ex12.php: registre três leituras sucessivas e uma leitura inválida, verificando o consumo em cada período.

Exercício 13 - Máquina de snacks [Preparação para prova]

Classe: `MaquinaSnack`

Propriedades:

- public string $produto
- private float $preco
- private int $estoque
- private float $credito

Métodos e retornos:

- reabastecer(int $quantidade): bool - aumenta o estoque com quantidade positiva.
- inserirCredito(float $valor): bool - aumenta o crédito com valor positivo.
- comprar(): bool - realiza a compra somente com estoque e crédito suficientes.
- devolverCredito(): float - retorna o crédito existente e zera o crédito da máquina.
- status(): string - informa produto, preço, estoque e crédito.

Regras:

- O estoque inicia em 0 e o crédito em 0.
- Uma compra válida reduz o estoque em 1 e o crédito pelo preço do produto.
- Compra inválida não pode alterar estoque nem crédito.
- O crédito restante pode permanecer para uma próxima compra ou ser devolvido.

Teste no ex13.php: teste falta de estoque, crédito insuficiente, compra bem-sucedida, compra com crédito excedente e devolução.

Exercício 14 - Bateria de dispositivo [Preparação para prova]

Classe: `BateriaDispositivo`

Propriedades:

- public string $dispositivo
- private int $carga

Métodos e retornos:

- usar(int $minutos): bool - consome 1% de carga a cada 5 minutos iniciados.
- carregar(int $percentual): int - adiciona carga e retorna quanto foi realmente acrescentado.
- nivel(): int - retorna a carga atual.
- estaCritica(): bool - retorna true quando carga <= 15.
- status(): string - retorna dispositivo e nível.

Regras:

- A carga deve permanecer entre 0 e 100.
- Não é possível usar o dispositivo quando a carga estiver em 0.
- Se o tempo solicitado exigir mais carga do que existe, consuma somente até zerar e retorne false.

Teste no ex14.php: inicie com uma carga intermediária, faça usos de diferentes durações, carregue acima de 100% e confirme o valor efetivamente adicionado.

Exercício 15 - Drone de inspeção [Preparação para prova]

Classe: `DroneInspecao`

Propriedades:

- public string $modelo
- private int $bateria
- private float $distanciaTotal
- private bool $emVoo

Métodos e retornos:

- decolar(): bool - permite decolagem somente se bateria >= 20 e o drone não estiver voando.
- voar(float $km): bool - consome 5% de bateria por km e soma a distância percorrida.
- pousar(): bool - encerra o voo quando o drone estiver em voo.
- recarregar(int $percentual): int - aumenta a bateria sem ultrapassar 100.
- status(): string - retorna bateria, distância e situação do voo.

Regras:

- bateria deve permanecer entre 0 e 100.
- Não é possível voar sem decolar.
- Se a bateria não for suficiente para toda a distância pedida, percorra apenas a distância possível antes de zerar.

Teste no ex15.php: simule decolagem com pouca bateria, recarga, voo válido, tentativa de recarga acima do limite e pouso.

Exercício 16 - Pacote em rota de entrega [Transição]

Uma transportadora acompanha pacotes entre o centro de distribuição e o destinatário. Cada pacote pode sair para entrega algumas vezes, pode falhar por ausência do destinatário e, depois de três tentativas sem sucesso, deve retornar ao remetente. O sistema precisa impedir transições impossíveis de estado.

Exercício de transição: a estrutura principal é indicada, mas as validações e mudanças de estado precisam ser interpretadas a partir do cenário.

Classe: `PacoteEntrega`

Propriedades:

- public string $codigo
- public string $destino
- private string $status
- private int $tentativas

Métodos e retornos:

- sairParaEntrega(): bool - inicia uma tentativa válida de entrega.
- registrarFalha(): bool - registra falha quando o pacote estiver em rota.
- confirmarEntrega(): bool - marca como entregue quando o pacote estiver em rota.
- statusAtual(): string - retorna status e tentativas.

Regras:

- O pacote inicia com status "aguardando" e zero tentativas.
- Cada saída para entrega aumenta o número de tentativas.
- Após três tentativas sem sucesso, o pacote deve entrar automaticamente em "devolução".
- Pacotes entregues ou em devolução não podem iniciar uma nova tentativa.
- Uma falha só pode ser registrada se houver uma tentativa em andamento.

Teste no ex16.php: crie cenários de entrega na primeira tentativa, falhas sucessivas e bloqueio de uma quarta tentativa.

Desafios de interpretação e modelagem

Nos exercícios 17 a 20, o enunciado não informa a classe, as propriedades ou os métodos. O objetivo é praticar abstração: leia o cenário, identifique o que pertence ao estado do objeto, o que deve ser protegido com private e quais operações públicas preservam as regras. Não utilize recursos de POO ainda não estudados.

Exercício 17 - Smart Locker de encomendas [Desafio]

Um condomínio instalou armários inteligentes para receber encomendas. Cada compartimento começa livre. Ao receber um pacote, o sistema registra uma identificação da encomenda e um código numérico de retirada. Enquanto estiver ocupado, outro pacote não pode ser colocado no mesmo compartimento. A retirada só deve ocorrer com o código correto. Após três códigos incorretos consecutivos, o compartimento fica bloqueado e não aceita novas tentativas até que seu estado seja redefinido por uma operação apropriada. Depois de uma retirada válida, o compartimento volta a ficar livre.

Desafio de modelagem: a partir do cenário, identifique a classe necessária, as propriedades, quais delas devem ser privadas, os comportamentos públicos e as regras que protegem o estado do objeto.

Requisitos mínimos:

- Utilize pelo menos uma classe.
- Proteja com private o estado que não pode ser alterado livremente.
- Use property promotion onde fizer sentido.
- Implemente retornos claros para operações válidas e inválidas.

Teste no ex17.php: simule depósito de uma encomenda, tentativas de retirada com código incorreto, bloqueio após três erros e uma retirada válida em um novo objeto.

Exercício 18 - Plano de dados móvel [Desafio]

Uma operadora quer representar um plano pré-pago de internet móvel. O cliente possui uma franquia de dados medida em GB. O sistema registra o consumo ao longo do mês e permite comprar pacotes adicionais. Um consumo só pode ser confirmado quando houver dados suficientes. Quando a franquia chegar a zero, novas tentativas de uso devem ser recusadas até que um pacote adicional seja contratado. O histórico total consumido no mês também precisa continuar coerente.

Desafio de modelagem: a partir do cenário, identifique a classe necessária, as propriedades, quais delas devem ser privadas, os comportamentos públicos e as regras que protegem o estado do objeto.

Requisitos mínimos:

- Modele o estado do plano sem permitir consumo negativo ou saldo de dados abaixo de zero.
- O objeto deve ser responsável por decidir se um consumo pode ser realizado.
- Deve existir uma forma de consultar o saldo restante e a situação do plano.
- Use public e private de forma intencional.

Teste no ex18.php: crie ao menos dois planos com franquias diferentes, simule consumos válidos, consumo maior que o saldo e compra de pacote adicional.

Exercício 19 - Robô de arena [Desafio]

Em um jogo de arena, cada robô possui nome, energia, integridade estrutural e uma pontuação de desempenho. Treinar aumenta a pontuação, mas consome energia. Participar de um combate consome ainda mais energia e provoca desgaste estrutural. Um robô sem energia suficiente ou com integridade zerada não pode entrar em combate. Há uma estação de recarga e uma oficina de reparo, mas nenhum valor pode ultrapassar os limites máximos definidos pelo próprio robô. Modele esse comportamento protegendo o estado interno contra alterações diretas.

Desafio de modelagem: a partir do cenário, identifique a classe necessária, as propriedades, quais delas devem ser privadas, os comportamentos públicos e as regras que protegem o estado do objeto.

Requisitos mínimos:

- A energia e a integridade do robô devem possuir limites claros.
- Ações devem alterar o estado somente quando seus pré-requisitos forem atendidos.
- O aluno deve definir pelo menos quatro comportamentos públicos.
- O teste deve demonstrar pelo menos duas operações recusadas por regras do objeto.

Teste no ex19.php: crie dois robôs com estados iniciais diferentes e simule uma sequência de treino, combate, reparo e recarga até chegar a situações-limite.

Exercício 20 - Sobrevivência em Marte [Desafio]

Um jogo de sobrevivência em Marte precisa controlar um astronauta durante uma missão. Ele possui níveis de vida, energia e oxigênio. Explorar uma região consome energia e oxigênio; descansar recupera energia, mas não oxigênio; utilizar um cilindro recupera oxigênio, desde que ainda existam cargas disponíveis. Nenhum nível pode ser negativo ou ultrapassar seu máximo. Se a vida ou o oxigênio chegar a zero, o astronauta não pode continuar explorando. O sistema deve manter essas regras mesmo que o código de teste tente executar ações em uma ordem inadequada.

Desafio de modelagem: a partir do cenário, identifique a classe necessária, as propriedades, quais delas devem ser privadas, os comportamentos públicos e as regras que protegem o estado do objeto.

Requisitos mínimos:

- Identifique quais dados representam estado permanente e quais são apenas resultados calculados.
- Crie métodos que expressem ações do personagem em vez de permitir alteração direta dos atributos.
- Defina limites e valide todas as transições importantes.
- A solução deve permitir consultar um resumo do estado sem expor a possibilidade de modificá-lo diretamente.

Teste no ex20.php: simule uma sequência de exploração, alimentação/recuperação e descanso que leve o personagem a pelo menos uma condição crítica e depois o recupere.

Checklist antes de enviar

- Existem arquivos de teste separados de ex1.php até ex20.php para os exercícios realizados.
- Cada classe está em arquivo próprio e com nome coerente.
- Cada arquivo exN.php possui o comentário com nome, RA, turma e disciplina.
- As propriedades private não são alteradas diretamente nos arquivos de teste.
- Os métodos públicos são responsáveis por validar e proteger as regras do objeto.
- Property promotion foi usada quando adequada e sem transformar todas as propriedades em public apenas por conveniência.
- Os testes incluem casos válidos e inválidos e demonstram claramente os retornos dos métodos.
- O projeto executa sem erros no PHP 8.4.
- A pasta final foi compactada em .zip e está pronta para envio no Moodle.

Uma classe bem encapsulada não apenas guarda dados: ela impede que o objeto chegue a estados que não fazem sentido.